

Probabilistic Analysis & Quick Sort

Introduction

This technical note summarize the tool and logic to use in probabilistic analysis.

Type of probabilistic analysis

There are 4 different versions about the probabilistic analysis

- Random if
 - With Recursion
 - Without recursion
- Random for
 - single random loop
 - Random loop inside an outer loop
- Random-sized recursive call (i.e. Number axis problem)
- Loop that randomly terminates

1. Random if

Method: *Find the probability of both condition and their $T(n)$, then use the fomula*

1.1 Random if with recursion

```

1. k = Random(n)
2. if Boolean condition on k then
3.     Make recursive call
4. else
5.     Make recursive call
6. end if

```

For example

```

function Function1(A[], n) {
    k = random(n)
    if (k mod 4 = 1) {
        x = x + Function1(A[], n/2)
    } else {
        x = x + Function1(A[], n/3)
    }
}

```

We can use the following fomula:

$$ET(n) = \Pr(A) * E(T(n) | A) + \Pr(Not A) * E(T(n) | Not A)$$

From the fomula, we can easily get that

$$ET(n) = \frac{1}{4} * T(\frac{n}{2}) + (1 - \frac{1}{4}) * T(\frac{n}{3})$$

1.2 Random if without recursion

```

1. k = Random(n)
2. if Boolean condition on k then
3.     Foo()
4. else
5.     Bar()
6. end if

```

For example

```

function Function2(A[], n) {
    k = random(n)
    int x = 0;
    if (k mod 4 = 1) {
        for (i = 1 to n) {
            x = x + A[i];
        }
    } else {
        int i = 1;
        while (i <= n) {
            x = x + A[i];
            i = 2 * i;
        }
    }
}

```

We can use the following formula (Same as 1.1):

$$ET(n) = \Pr(A) * E(T(n) \mid A) + \Pr(\text{Not } A) * E(T(n) \mid \neg A)$$

OR

$$ET(n) = \Pr(A) * E(T(n) \mid A) + \Pr(\text{Not } A) * E(T(n) \mid \text{NOT } A)$$

OR

$$ET(n) = \Pr(\text{Foo}) * E(T(n) \mid \text{Foo}) + \Pr(\text{Bar}) * E(T(n) \mid \text{Bar})$$

Since

$$\Pr(A) = \frac{1}{4}, E(T(n) \mid \text{Foo}) = \Theta(n)$$

and

$$\Pr(\neg A) = \frac{3}{4}, E(T(n) \mid \text{Bar}) = \Theta(\log n)$$

From the formula, we can easily get that

$$ET(n) = \frac{1}{4} * \Theta(n) + \frac{3}{4} * \Theta(\log n)$$

2. Random for

Method: First handle the overall $T(n)$ with the random k and then plug in the result to the for loop if there is one.

2.1 single random loop

1. $k = \text{Random}(n)$
2. Loop that runs $f(k)$ times

For example

```
function Function3(A[], n) {
  x = 0;
  k = Random(n);
  for (i = 1 to k){
    for(j = 1 to i * i) {
      x = x + A[ij mod n];
    }
  }

  return x;
}
```

We can use the formula:

$$ET(n) = \sum_{q=1}^n \Pr(k = q)t(k = q)$$

We should get

$$t(k = q)$$

first. It is the loop under the k

$$t(k = q) = \left(\sum_{i=1}^q i^2 C \right)$$

Since k is uniformly random,

$$\Pr(k = q) = \frac{1}{n}$$

Therefore,

$$ET(n) = \sum_{q=1}^n \frac{1}{n} \sum_{i=1}^q i^2 C$$

2.2 Random loop inside an outer loop

```
1. loop
2.     k = Random(i)
3.     Loop that runs f(i) times
4. end loop
```

For example

```
function Function4(A[], n) {
    x = 0;
    for (i = 1 to n) {
        k = Random(i);
        for (j = 1 to k * k){
            x = x + A[(ij mod n)]
        }
    }

    return x;
}
```

For each iteration of the outer loop, we solve the time complexity of inner loop first.

We can use the formula:

$$ET_i(n) = \sum_{q=1}^i \Pr(k = q) t(k = q)$$

Since k is uniformly random from 1 to i ,

$$\Pr(k = q) = \frac{1}{i}$$

and because the inner loop runs k^2 times,

$$t(k = q) = q^2 C$$

Therefore,

$$\begin{aligned} ET_i(n) &= \sum_{q=1}^i \frac{1}{i} \Theta(q^2) \\ &= \frac{1}{i} \Theta(i^3) = \Theta(i^2) \end{aligned}$$

Then, we can find the overall time complexity

$$ET(n) = \sum_{i=1}^n ET_i(n)$$

$$= \sum_{i=1}^n \Theta(i^2) = \Theta(n^3)$$

3. Random-sized recursive call (i.e. Number axis problem)

Method: *First figure out the best and the worst cases, then use the number axis to divide the condition, then use the best/worst case in the division to get the overall time complexity.*

1. bunch of base case and normal loop
2. Do ()
3. k = Random(n)
4. Make recursive call of size f(k).
4. return

For example

```
function Function5(A[], n) {
    if (n <= 22) {
        return 1;
    }
    x = 0;
    for (i = 1 to n/2) {
        for (j = 1 to n/4) {
            x = x + A[(ij mod n)];
        }
    }
    k = Random(n/5 - 1);
    x = x + Func5(A[], 5k);
    x = x + Func5(A[], n - 5k);

    return x;
}
```

We use the following steps:

1. Substitute the minimum and maximum possible values of k to identify the worst-case recursive-call sizes.
2. Find the midpoint where the two recursive calls have the same size.
3. Divide the possible values of k into several ranges using the critical points on the number line.
4. For each range, choose the value of k that gives the worst recursive-call sizes, then use these cases to construct the upper bound.
5. The lower bound is the same. Just use best case

Solution:

At the two endpoints,

$$k = 1 \Rightarrow T(5) + T(n - 5)$$

and

$$k = \frac{n}{5} - 1 \Rightarrow T(n - 5) + T(5).$$

Both endpoints give the same unbalanced recursive-call sizes.

To find the midpoint, set the two recursive-call sizes equal:

$$5k = n - 5k$$

$$10k = n$$

$$k = \frac{n}{10}.$$

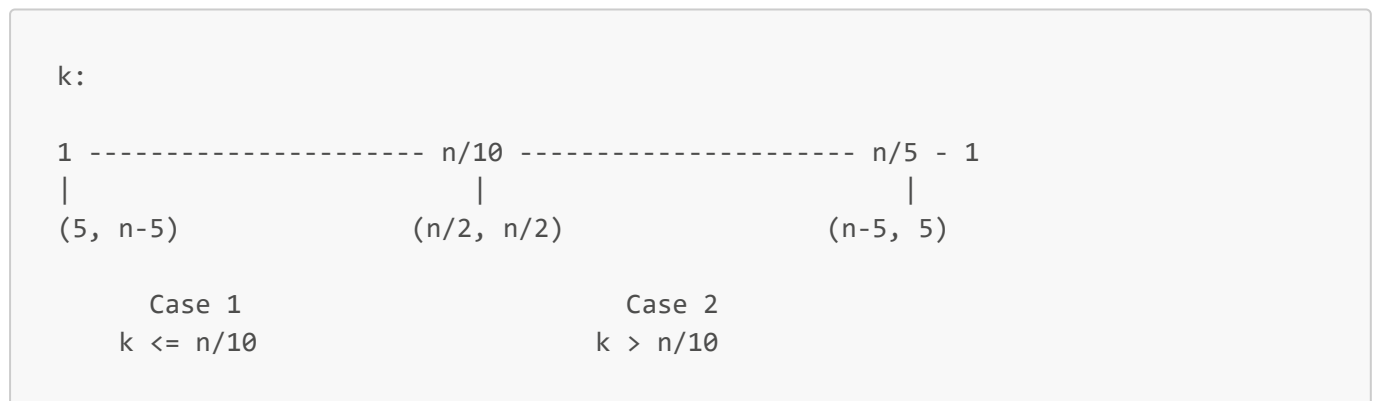
Therefore,

$$5k = n - 5k = \frac{n}{2},$$

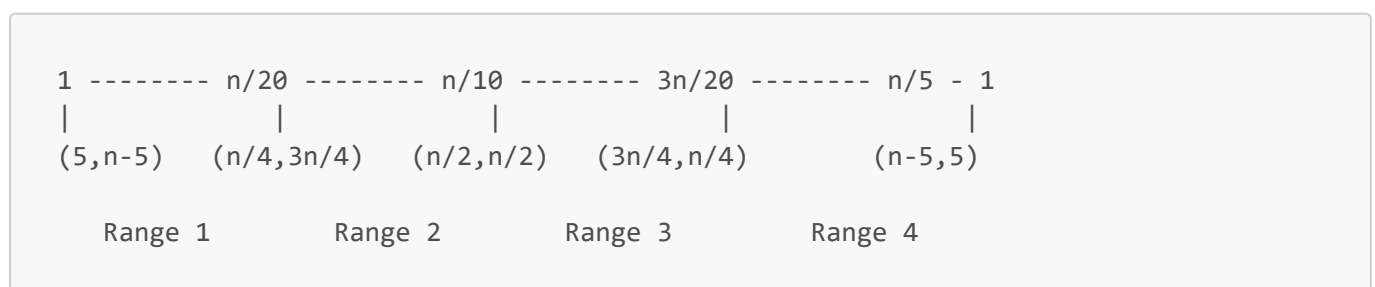
so the recursive cost at the midpoint is

$$2T(n/2).$$

Then we draw the number line:



Next, divide the possible values of k into four ranges:



For the upper bound, choose the worst recursive-call sizes in each range.

Therefore,

$$ET(n) \leq \Theta(n^2) + \frac{1}{4} [ET(5) + ET(n-5)] + \frac{1}{4} [ET(n/4) + ET(3n/4)] \\ + \frac{1}{4} [ET(3n/4) + ET(n/4)] + \frac{1}{4} [ET(n-5) + ET(5)].$$

Combining identical terms gives

$$ET(n) \leq \Theta(n^2) + \frac{1}{2} [ET(5) + ET(n-5)] + \frac{1}{2} [ET(n/4) + ET(3n/4)].$$

Since $ET(5) = \Theta(1)$, it can be absorbed into the other asymptotic terms.

For the upper bound, We can also replace $ET(n-5)$ by $ET(n)$ because

$$n-5 < n$$

and therefore

$$ET(n-5) \leq ET(n).$$

Thus,

$$ET(n) \leq \Theta(n^2) + \frac{1}{2} ET(n) + \frac{1}{2} ET(n/4) + \frac{1}{2} ET(3n/4).$$

Move $\frac{1}{2} ET(n)$ to the left-hand side:

$$ET(n) - \frac{1}{2} ET(n) \leq \Theta(n^2) + \frac{1}{2} ET(n/4) + \frac{1}{2} ET(3n/4).$$

Therefore,

$$\frac{1}{2} ET(n) \leq \Theta(n^2) + \frac{1}{2} ET(n/4) + \frac{1}{2} ET(3n/4).$$

Multiplying both sides by 2 gives

$$ET(n) \leq ET(n/4) + ET(3n/4) + \Theta(n^2).$$

Therefore,

$$ET(n) = O(n^2).$$

For the Lower Bound

The expected running time cannot be better than the best possible recursive split.

The best split occurs at

$$k = \frac{n}{10},$$

which gives two recursive calls of size $n/2$.

Therefore, the best-case recurrence is

$$T(n) = 2T(n/2) + \Theta(n^2).$$

This recurrence gives

$$T(n) = \Theta(n^2).$$

Since the expected running time cannot be better than the best-case running time,

$$ET(n) = \Omega(n^2).$$

Therefore,

$$ET(n) = \Theta(n^2)$$

4. Loop that randomly terminates

Reference List

1. "CSE2331_Types_of_Probabilistic_Problems" Created by Professor Painter.
2. "Quick Sort Worksheet AU26" Created by Professor Painter.
3. "CSE2331_QuickSort_and_Probabilistic_Analysis_Homework" Created by Professor Painter.